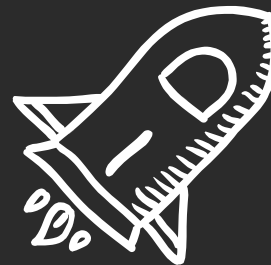




Semana 14 ¡Bienvenidos!



Temas de hoy

1 Plantillas en Django

3 Uso de contextos

2 Herencia de plantillas

1

Plantillas en Django

Es el momento de adentrarnos en nuestro Front-end. Hasta el momento solo sabemos utilizar HTML y CSS para dar una forma visual agradable a nuestro sitio web, pero solo vimos cómo insertar información estática, es decir, que la información que mostremos debemos agregarla a mano y no cambiará.

Django nos tiene preparado una solución 😊

Introducción a las Plantillas de Django

¿Qué son?

Las plantillas en Django son los archivos HTML que estructuran el diseño de nuestra página web pero con una gran diferencia; Django le provee de un lenguaje específico de plantilla que nos va a permitir insertar datos dinámicamente en nuestros archivos HTML que enviaremos desde las aplicaciones de nuestro proyecto.

Ventajas

Las ventajas del uso de plantillas en Django van más allá de la simple generación de HTML, ya que se basan en principios sólidos de desarrollo de software que mejoran la calidad y sostenibilidad del código.

- **Separación de lógica:** Las plantillas te permiten manejar la presentación de los datos de forma independiente a la lógica que los procesa.
- **Reutilización de código:** Nos facilitan la reutilización de componentes comunes, sin tener que duplicar el código en cada página, haciendo uso de la herencia de plantillas que nos permite Django.
- **Facilidad de mantenimiento:** Realizar cambios en el diseño es más sencillo, debido a que cada parte del sitio estará encapsulada en bloques reutilizables y modularizados.
- **Flexibilidad en la presentación de datos:** Nos permite utilizar etiquetas y filtros para transformar y presentar datos de manera dinámica y flexible.
- **Integración con seguridad:** Viene integrado con funcionalidades de seguridad como el escape automático de variables para evitar ataques XSS (Cross-Site Scripting), y etiquetas para proteger formularios de ataques CSRF.

Sistema de Plantillas

El sistema de Plantillas del que hablamos anteriormente es el “lenguaje” que Django nos provee para poder manipular y mostrar los datos en un HTML de forma dinámica, debido a que los archivos HTML por si mismos no tienen la capacidad de resolverlos.

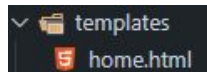
Este sistema de plantillas nos ofrece:

- Una sintaxis sencilla la cual es fácil de aprender e implementar.
- Etiquetas y filtros los cuales serán los encargados de tratar directamente los datos en la plantilla.
- Seguridad contra inyecciones de código malicioso.
- Una extensibilidad ya que se pueden crear etiquetas y filtros personalizados.

Veamos sus componentes principales y cómo implementarlos.

Para empezar a trabajar, precisamos crear un archivo con extensión “.html” en nuestra carpeta “templates” de nuestro proyecto.

Más adelante hablaremos de cómo enviaremos los datos desde las vistas a las plantillas utilizando contextos, pero de momento vamos a centrarnos en cómo recibiremos y dispondremos esos datos en nuestras plantillas.



```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Home</title>
</head>

<body>

</body>

</html>
```

Sistema de Plantillas: Variables

Variables

El sistema de plantillas nos deja recibir y utilizar variables dentro de un HTML siguiendo una estructura definida: “{{}}”

```
<body>

  <h1>Estas cursando actualmente: {{curso}} </h1>

</body>
```

Por ejemplo, nosotros dentro de nuestra plantilla estamos definiendo un título (<h1>) que dice “Estás cursando actualmente:”. Este texto se encuentra estático y no se modificará bajo ninguna condición, sin embargo nos encontramos con {{curso}} donde las llaves “{{}}” es la sintaxis que ocupa nuestro sistema de plantillas para definir que la palabra que se encuentra dentro es una variable.

Por lo tanto Django trabajará ese dato “{{curso}}” de forma dinámica dependiendo del valor que tenga curso.

En nuestro caso, supongamos que la variable “curso” recibe como valor “Desarrollo Web (Etapa 2)”.

Dandonos como resultado:



En conclusión, dependiendo del valor que reciba nuestra variable “{{curso}}”, la etiqueta <h1> se modifica parcialmente pero dinámicamente acomodándose a nuestras necesidades.

Sistema de Plantillas: Etiquetas (if, else)

Etiquetas

Al igual que podemos manipular variables dentro de nuestras plantillas, también tenemos la capacidad de realizar lógica utilizando estructuras condicionales y de control dentro de la misma siguiendo la sintaxis “{% %}”.

Supongamos que recibimos en nuestra plantilla una variable “usuario” que almacena el siguiente diccionario:

```
usuario = {  
  "nombre": "Infor",  
  "apellido": "Matorio",  
  "edad": 32,  
  "estado_civil": "Soltero",  
  "titulo_universitario": True  
}
```

```
<body>  
<ul>  
  <li>El usuario es: {{usuario.nombre}} {{usuario.apellido}}</li>  
  
  {% if usuario.edad >= 18 %}  
    <li>Es mayor de edad {{usuario.edad}} años</li>  
  {% else %}  
    <li>Es menor de edad</li>  
  {% endif %}  
  
  <li>Estado civil: {{usuario.estado_civil}}</li>  
  
  {% if usuario.titulo_universitario %}  
    <li>Posee un titulo universitario</li>  
  {% else %}  
    <li>No tiene estudios universitarios</li>  
  {% endif %}  
</ul>  
</body>
```

Creamos una lista de items para mostrar la información del usuario que recibimos, y utilizando nuestra nueva sintaxis podemos definir estructuras condicionales como lo es “if” y “else” donde damos la posibilidad a nuestra plantilla de decidir cual etiqueta HTML renderizar. Por ejemplo, la primer condición verificamos que el usuario sea mayor de edad para mostrar un mensaje acorde al resultado.

Sistema de Plantillas: Etiquetas (if, else)

Por otro lado, en el segundo condicional verificamos si posee un título universitario y renderizamos un mensaje acorde de tenerlo o no.

Otra particularidad importante que tiene esta sintaxis es que los condicionales requieren obligatoriamente de una etiqueta de cierre como lo podemos observar al final de cada condicional que utilizamos `{% endif %}` para cerrar el bloque.

Entonces nosotros recibimos esta información en nuestra plantilla:

```
usuario = {  
  "nombre": "Infor",  
  "apellido": "Matorio",  
  "edad": 32,  
  "estado_civil": "Soltero",  
  "titulo_universitario": True  
}
```

Y gracias a la lógica implementada en nuestra plantilla terminamos renderizando esto:



- El usuario es: Infor Matorio
- Es mayor de edad (32 años)
- Estado civil: Soltero
- Posee un título universitario

Pero no está de más aclarar que además de permitarnos tomar decisiones en base a estos datos específicos que recibimos, también funcionará para cualquier usuario que pasemos su información, sea mayor o menor de edad, o tenga o no título universitario. Tendrá un mensaje acorde a cualquier usuario que llegue como dato a nuestra plantilla.

Pero entonces, ¿en el caso de que recibamos una lista de varios usuarios? ¿cómo hacemos?

Sistema de Plantillas: Etiquetas (for)

En el caso de que queramos manipular una lista de datos, como ya lo anticipamos, también podemos utilizar estructuras de control para ello.

Por ejemplo supongamos que nosotros recibimos en una variable “usuarios” la siguiente lista cuyos elementos son diccionarios con la información de cada usuario 🤖

```
usuarios = [  
  {  
    "nombre": "Maria",  
    "apellido": "Perez",  
    "edad": 16,  
    "estado_civil": "Soltero",  
    "titulo_universitario": True  
  },  
  {  
    "nombre": "Jose",  
    "apellido": "Gomez",  
    "edad": 21,  
    "estado_civil": "Casado",  
    "titulo_universitario": False  
  },  
  {  
    "nombre": "Martin",  
    "apellido": "Toledo",  
    "edad": 42,  
    "estado_civil": "Divorciado",  
    "titulo_universitario": True  
  },  
]
```

Elemento 1

Elemento 2

Elemento 3

Nuestro objetivo es mostrar un mensaje igual al ejemplo anterior pero por cada uno de los usuarios de la lista. Para ello, implementaremos la etiqueta “for” que recorrerá cada uno de los elementos:

```
{% for usuario in usuarios %}  
  <ul>  
    <li>El usuario es: {{usuario.nombre}} {{usuario.apellido}}</li>  
  
    {% if usuario.edad >= 18 %}  
      <li>Es mayor de edad ({{usuario.edad}} años)</li>  
    {% else %}  
      <li>Es menor de edad</li>  
    {% endif %}  
  
    <li>Estado civil: {{usuario.estado_civil}}</li>  
  
    {% if usuario.titulo_universitario %}  
      <li>Posee un titulo universitario</li>  
    {% else %}  
      <li>No tiene estudios universitarios</li>  
    {% endif %}  
  </ul>  
{% endfor %}
```

Sistema de Plantillas: Etiquetas (for)

Por lo tanto, recorreremos nuestra variable usuarios y en cada iteración renderizará toda nuestra estructura HTML pero aplicado a cada usuario en la lista.

Dandonos como resultado:

- El usuario es: Maria Perez
- Es menor de edad
- Estado civil: Soltero
- Posee un título universitario
- El usuario es: Jose Gomez
- Es mayor de edad (21 años)
- Estado civil: Casado
- No tiene estudios universitarios
- El usuario es: Martin Toledo
- Es mayor de edad (42 años)
- Estado civil: Divorciado
- Posee un título universitario

Nuevamente este ejemplo nos permite llegar a las mismas conclusiones.

Sin importar el usuario, nuestra plantilla está preparada de tal manera que aunque sus datos difieran, pueda resolver y renderizar un mensaje acorde a los datos.

Además sin darnos cuenta, redujimos a un tercio la cantidad de líneas de código que necesitaríamos para lograr el mismo resultado sin este sistema de plantillas, ya que reutiliza la misma estructura HTML para los 3 usuarios.

Sistema de Plantillas: Filtros

Filtros

Por último tenemos la posibilidad de aplicar filtros a nuestros datos que ingresan a la plantilla las cuales modificarán el valor de la variable.

Si bien se mostrará un par de ejemplos, insistimos de que investiguen y encuentren todos los filtros que Django les proporciona.

- Si tenemos una variable de tipo fecha (Datetime) podemos modificar su formato:

```
<p>{{ fecha|date:"F j, Y" }}</p>
```

```
<p>{{ fecha|time:"H:i" }}</p>
```



octubre 4, 2024

14:30

- Si deseamos acortar el texto que recibimos en una variable para mostrar una cierta cantidad de caracteres:

```
<p>{{ titulo|truncatechars:25 }}</p>
```



Bienvenidos a nuestra 2d...

- Si en un string utilizamos “\n” en los lugares donde nos gustaría que haya un salto de línea, podremos ocupar un filtro que lo divida en párrafos diferentes. Si nuestra plantilla recibe una variable como esta:

```
texto = "Este es mi primer párrafo.\nEste es mi segundo párrafo"
```

```
<p>{{ texto|linebreaks }}</p>
```



Este es mi primer párrafo.
Este es mi segundo párrafo

Sistema de Plantillas: Filtros

- En el caso de querer definir la cantidad de decimales que pueden tener los números en tu plantilla, se puede ocupar:

```
<p>{{ num_decimal|floatformat:2 }}</p>
```



4,68

- También se puede incrementar el valor de una variable. Si mi contador de likes de una publicación ya tenía **5** inicialmente, puedo ocupar:

```
<p>{{ likes|add:1 }}</p>
```



Likes: 6

- Si recibimos una lista de elementos y queremos crear un solo string con todos los valores.

Si recibo una lista como esta en mi plantilla:

```
lista_frutas = ["Banana", "Manzana", "Pera", "Limon"]
```

Puedo utilizar el filtro "join:"

```
<p>{{ lista_frutas|join:", " }}</p>
```



Banana, Manzana, Pera, Limon

Nótese que debemos especificar entre las comillas cual será el carácter que separe cada uno de los elementos. En este ejemplo es una coma (,) y un espacio vacío.

2

Herencia de plantillas

Basándonos en el concepto de Herencia que aprendimos en Programación Orientada a Objetos, las plantillas tienen la capacidad para crear un archivo HTML base con ciertas características de la cual heredaremos (extenderemos) las demás plantillas para compartir entre todas las mismas particularidades.

Veamoslo más detalladamente.

Herencia de plantillas

La herencia de plantillas nos permite a nosotros crear una primer plantilla base la cual contendrá la estructura principal de nuestro sitio, para que luego otras plantillas puedan extender y personalizar esta primer plantilla base.

Esta manera de trabajar las plantillas promueve la reutilización y consistencia de nuestro diseño del sitio web.

Ventajas:

- Mantiene una apariencia uniforme en todas las páginas de nuestro sitio dándonos consistencia en el diseño.
- Al hacer cambios en la plantilla base, podremos ver reflejado automáticamente en todas las plantillas derivadas facilitando ampliamente el mantenimiento de las mismas.
- Como ya lo comentamos en reiteradas ocasiones, evita la duplicación de código innecesario fomentando la reutilización de código.

Herencia de plantillas: Implementacion

Para aplicar herencia en nuestras plantillas primero debemos crear nuestra plantilla padre o base.

En esta plantilla buscamos que contenga los componentes que se repetirán obligatoriamente en todas las demás páginas.

Por ejemplo, el “header” y el “footer” son estructuras que no suelen cambiar al navegar por diferentes páginas dentro de un sitio.

Al igual que nuestro bloque “head” y “html” que son vitales para el desarrollo de archivos HTML.

Al haber nosotros encontrado un patrón de código que se repetiría en todas nuestras páginas, vamos a desarrollar esa estructura en nuestra plantilla base.

Como por ejemplo:

```
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <meta charset="UTF-8">
6   <title>Mi blog de noticias</title>
7 </head>
8
9 <body>
10   <header>
11     <h1>INFORMATARIO - ETAPA 2</h1>
12     <nav>
13       <ul>
14         <li><a href="/">Inicio</a></li>
15         <li><a href="/about/">Acerca de</a></li>
16       </ul>
17     </nav>
18   </header>
19   <hr>
20   <br>
21
22   <main>
23
24   </main>
25
26   <br>
27   <hr>
28   <footer>
29     <p>©copy; 2024 Mi Sitio Web</p>
30   </footer>
31 </body>
32
33 </html>
```

En este ejemplo creamos un archivo en la carpeta templates llamado base.html donde definimos la estructura principal obligatoria de un archivo HTML, y agregamos lo que sería nuestro header y footer. En este caso el header tiene una barra de navegación con links que me llevan a las página de “Inicio” y “Acerca de”.

Pero como podemos observar, nuestra etiqueta `<main>` se encuentra vacía. Veamos por qué 🙋

Herencia de plantillas: Implementacion

Hasta ahora en nuestro archivo base.html definimos la estructura HTML que heredarán las demás plantillas.

Pero como pudimos notar en el ejemplo, dentro de nuestras etiquetas `<main>` que es donde debería estar la información que queremos mostrar de cada página, de momento se encuentra vacío.

Es aquí donde implementamos el primer paso para poder utilizar herencia, definiendo un bloque de contenido de la siguiente manera:

```
<main>

    {% block content %}

    <!-- Contenido específico de la página -->

    {% endblock %}

</main>
```

`{% block content %}` y `{% endblock %}` nos permite a nosotros definir dónde y qué contenido se modificará dependiendo de la página que estemos renderizando.

Es decir, que en donde se encuentran ambas etiquetas será el espacio que cambiará dinámicamente entre páginas, haciendo así que el resto de la estructura HTML permanezca intacta de modificaciones y evitándonos repetir la misma estructura en cada una de nuestras plantillas.

De esta manera nosotros terminamos de crear nuestra plantilla base (base.html).

Ahora veamos como podemos heredar de ella 👉

Herencia de plantillas: Implementacion

Supongamos que ahora queremos crear la plantilla de nuestra página de inicio.

Para ello primero crearemos nuestro archivo inicio.html y deberemos “extender” del “base.html”:

```
inicio.html X
templates > inicio.html > ...
1  {% extends "base.html" %}
```

Eso lo que hará es heredar toda la estructura de nuestro base.html.

Por lo tanto, nuestro siguiente paso será programar el contenido de nuestra página de inicio dentro de las etiquetas `{% block content %}` para que personalice nuestro archivo base.

```
inicio.html
templates > inicio.html > ...
1  {% extends "base.html" %}
2
3  {% block content %}
4
5      <h2>Bienvenido a la Página de Inicio</h2>
6      <p>Aca desarrollamos todas las secciones</p>
7      <p>que correspondan a nuestra página principal</p>
8      <p>Por ejemplo información importante sobre mi blog 🧡</p>
9
10 {% endblock %}
```

Dandonos como resultado:

INFORMATARIO - ETAPA 2

- [Inicio](#)
- [Acercas de](#)

Bienvenido a la Página de Inicio

Aca desarrollamos todas las secciones
que correspondan a nuestra página principal

Por ejemplo información importante sobre mi blog 🧡

Podemos notar que el header y el footer se mantuvieron de nuestro base.html, y el código que hicimos en inicio.html se agregó en el lugar donde definimos el `{% block content %}`, o sea en el `<main>`.

Herencia de plantillas: Implementacion

Ahora hagamos la segunda página de nuestra barra de navegación: “Acerca de”.

Para ello nuevamente extenderemos de base.html y desarrollaremos lo que mostraremos en esta página en específico:

```
acercaDe.html x
templates > acercaDe.html > ...
1  {% extends "base.html" %}
2
3  {% block content %}
4
5      <h2>Acerca de Nosotros</h2>
6      <p>Información sobre los integrantes del proyecto</p>
7      <ul>
8          <li>Diseñador: Lucas Fernández</li>
9          <li>Desarrollador Front-end: Mariana Gutiérrez</li>
10         <li>Desarrollador Back-end: Joaquín Ramos</li>
11         <li>Desarrollador Back-end: Valentina Ortiz</li>
12         <li>Arquitecto de base de datos: Daniel Acosta</li>
13         <li>Desarrollador QA: Sofía Méndez</li>
14     </ul>
15
16  {% endblock %}
```

Entonces realizamos el código HTML dentro del “block content” que se reemplazará al extenderse de base.html.

Dandonos como resultado:

INFORMATARIO - ETAPA 2

- [Inicio](#)
- [Acerca de](#)

Acerca de Nosotros

Información sobre los integrantes del proyecto

- Diseñador: Lucas Fernández
- Desarrollador Front-end: Mariana Gutiérrez
- Desarrollador Back-end: Joaquín Ramos
- Desarrollador Back-end: Valentina Ortiz
- Arquitecto de base de datos: Daniel Acosta
- Desarrollador QA: Sofía Méndez

3

Uso de Contexto

Hasta ahora estuvimos trabajando las plantillas asumiendo que recibiamos cierta información y a como manipularla.

Ahora nos centraremos a como enviar dicha información a nuestras plantillas, y para ello Django nos trae el concepto de “contexto”.

Contexto

Si bien el uso de contexto ya lo supimos definir anteriormente cuando empezamos a crear nuestras primeras vistas en el archivo `views.py`, todavía no tenía un uso real en nuestros ejemplos.

El contexto en Django es un diccionario que estaremos creando en nuestras vistas y enviándolo a nuestros templates con todos los datos que queramos que nuestra plantilla visualice.

Utilicemos el ejemplo del usuario que trabajamos anteriormente.

Primero debemos crear una vista “perfil” por ejemplo, y definiremos los datos de un usuario

```
def perfil(request):  
    data = {  
        "nombre": "Infor",  
        "apellido": "Matorio",  
        "edad": 32,  
        "estado_civil": "Soltero",  
        "titulo_universitario": True  
    }
```

Si hacemos memoria, para renderizar un template debemos retornar la función `render()` que nos permitía pasarle 3 argumentos: la `request`, el nombre del archivo `html` que queremos renderizar, y por último venía el contexto.

Es justamente en ese tercer argumento en donde pasaremos como un diccionario la información que al momento de renderizar el `html`, le proveerá de todos los datos que el contexto contiene:

```
def perfil(request):  
    data = {  
        "nombre": "Infor",  
        "apellido": "Matorio",  
        "edad": 32,  
        "estado_civil": "Soltero",  
        "titulo_universitario": True  
    }  
  
    return render(request, "perfil.html", { "usuario": data })
```

Nótese que en el diccionario la clave “usuario” será quien almacene toda la información que guarda “data”.

Contexto

Por lo tanto, cuando vayamos a nuestra url “perfil” y se ejecute la vista “perfil”, el template será capaz de mostrar la información que enviamos por el contexto, es decir, los datos del usuario:

perfil.html:

```
{% extends "base.html" %}

{% block content %}

<ul>
<li>El usuario es: {{usuario.nombre}} {{usuario.apellido}}</li>
<li>Tiene {{usuario.edad}} años.</li>
<li>Se encuentra {{usuario.estado_civil}}.</li>

  {% if titulo_universitario %}
    <li>Posee titulo universitario.</li>
  {% else %}
    <li>No cuenta con titulo universitario.</li>
  {% endif %}
</ul>

{% endblock %}
```

Nótese que utilizamos “usuario” como nombre de la variable porque es la clave que definimos en el contexto de la vista.

Dandonos como resultado:

INFORMATARIO - ETAPA 2

- [Inicio](#)
- [Acerca de](#)

- El usuario es: Infor Matorio
- Tiene 32 años.
- Se encuentra Soltero.
- No cuenta con titulo universitario

Contexto

Ahora que ya aprendimos a generar información y pasarla por el contexto, ¿se pueden imaginar el alcance que tiene esta poderosa posibilidad?

¿Y si en lugar de crear los datos manualmente, traemos información de nuestra base de datos y lo pasamos por contexto?

Trabajemos con un ejemplo que tienen ya conocido de clases anteriores como la de creación de eventos.

Primero vamos a crear nuestra vista “listar_eventos()” y en ella traeremos todos los eventos creados de nuestra base de datos utilizando la ORM, y la almacenamos en una variable “data”.

Luego pasemos toda esa información por contexto:

```
def listar_eventos(request):  
    data = Eventos.objects.all()  
    return render(request, 'eventos.html', {"eventos" : data})
```

Por último preparemos nuestro template “eventos.html” para que pueda recibir todos los eventos y visualizarlo correctamente.

Como podemos prever, en nuestra variable eventos puede haber más de un evento guardado, por lo tanto utilizaremos el ciclo for para poder mostrar cada uno de los eventos.

```
{% extends "base.html" %}  
  
{% block content %}  
    {% for evento in eventos %}  
  
        <hr>  
        <div>  
            <h3>{{evento.titulo}}</h3>  
            <p>Fecha del evento: {{evento.fecha}}</p>  
            <p>Ubicacion: {{evento.ubicacion}}</p>  
        </div>  
  
    {% endfor %}  
{% endblock %}
```

Contexto

Dandonos como resultado:

INFORMATARIO - ETAPA 2

- [Inicio](#)
- [Acercas de](#)

Mi primer evento

Fecha del evento: 3 Set. 2024

Ubicacion: Av Samiento 1242

Mi segundo evento modified

Fecha del evento: 28 Set. 2024

Ubicacion: Av Lavalle 9965

Mi tercer evento

Fecha del evento: 15 Enero 2025

Ubicacion: Av Paraguay y Av Italia

Mi cuarto evento

Fecha del evento: 15 Feb. 2024

Ubicacion: Av Marconi y Av McLean

Mi quinto y ultimo evento

Fecha del evento: 14 Feb. 2023

Ubicacion: Obligado 5918



Conclusión final

El uso de plantillas en Django es una herramienta muy poderosa que además de brindarnos muchas ventajas respecto a la creación de nuestro código, nos provee de funcionalidades que no son posibles con HTML nativo. Uniendo todos los conceptos aprendidos, nos podemos dar cuenta que poder guardar información en nuestra base de datos, para luego recuperarla y poder visualizarla en un template es una parte fundamental del ciclo de vida de nuestro sitio web.

Por lo tanto, con el manejo de la ORM, el uso del contexto, y las herramientas que nos da el sistema de plantillas para disponer la información de forma dinámica, da por finalizado nuestro ciclo completo, desde la request, hasta la response de nuestro proyecto.



**¡Nos vemos
En la próxima
clase!**

